

课程 COURSE

Big Data Analytics

大数据分析 [2026 国际商务]

Lecture 3 — Data Preparation 第三讲·数据整备
从数据质量到合并、清洗、缺失与面板整备

任课教师：戴谢尔 (Xieer Dai, David)

深圳大学 Shenzhen University · 第 8-14 周·星期四 11-12 节

上课地点：四方楼南 409

学分 2 · 选修课 (讲座式) | 答疑：预约制 | E-mail: Xieer.dai@szu.edu.cn

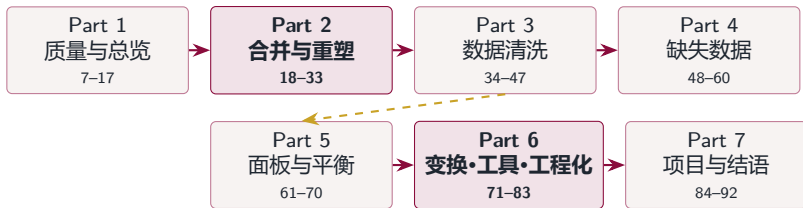
学习目标 Learning Objectives

完成本讲后，你应该能够：

1. 用有效性、准确性、完整性、一致性、单位一致性五项特征描述并量化一个数据集的质量；
2. 正确使用 `append / concat`（加观测）与 `merge / join`（加变量），判断连接基数并验证键的唯一性；
3. 在宽表与长表之间自由转换，熟练使用 `pivot / melt / stack / explode / cut` 等重塑工具；
4. 按清洗五步法处理重复、结构错误、异常值、缺失与验证，并留下可审计的清洗日志；
5. 识别 `MCAR / MAR / MNAR` 三种缺失机制，选择并评估插补策略，说清插补的代价；
6. 诊断面板的平衡性（T 是否一致、缺口在哪），完成变量变换并写出可复现的整备管道。

来源：沿用原课件 Lecture 3 Data Preparation 的四条主线（Join/Combine/Reshape、Data Cleaning、Missing Data、Tools），并按 2026 年工程实践扩展。

本讲路线图 Roadmap (92 帧)



主线：拿到数据 → **能用来建模的分析表**
→ 可复现、可审计

代码页 10 帧·样本页 10 帧：每页一份样本数据、一张可视化图、一段逐行英文注释的程序

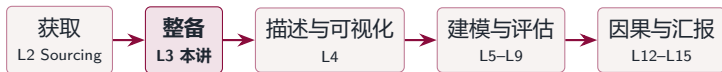
与上一讲的衔接 From Sourcing to Preparation

L2 结束时你手上有什么

- ▶ 一份数据源清单与可重跑的获取脚本；
- ▶ 若干原始文件：CSV、API 返回的 JSON、抓下来的网页表、PDF 抽出的表格；
- ▶ 但还没回答：这些数据能不能直接用？

L3 要回答的问题

- ▶ 数据质量如何量化？缺什么、错在哪？
- ▶ 多份数据怎么合并与重塑成一张分析表？
- ▶ 缺失、异常、重复该怎么处理才不留隐患？
- ▶ 怎么让整備过程可复现、可审计？



业界经验：数据分析项目中六到八成时间花在整備上——它不是“脏活”，而是结论可信度的来源。

本讲输出物 Deliverables

1) 清洗后的数据集

raw / curated 分层：原始快照不动，加工层可重建（本讲 11、14 页）

2) 可复现整备脚本

合并 → 清洗 → 缺失 → 变换，含断言与日志（代码页 16–74 页共 10 段）

3) 一页数据质量报告

五维评分 + 缺失/重复/异常/一致性量化结论 + 处理清单（86 页模板）

与本课程项目 1「数据获取与整备」对应

项目 1 的交付物正是这三件：**可复现的数据管道** + **数据说明** + **质量报告**。本讲 84–85 页给出要求与一页选题说明模板（第 6 周前提交：问题 / 数据 / 方法 / 预期产出）。

来源：评分与提交要求见课程大纲第六节；本讲提供报告模板与 30 项清单（86–87 页）。

全讲导览 Map of the Lecture

部分	帧	关键问题
数据质量与整備总览	7-17	什么算“干净”？怎么量化？粒度对不对？
合并与重塑	18-33	append 还是 merge？键唯一吗？宽表长表怎么换？
数据清洗	34-47	五步法怎么做？异常值该删还是留？
缺失数据	48-60	缺失是哪种机制？插补会不会制造假象？
面板与平衡性	61-70	T 一致吗？缺口怎么补、要不要补？
变换·工具·工程化	71-83	变换与标准化、单位统一、断言与监控怎么做？
项目与结语	84-92	项目 1 要交什么？常见错误有哪些？

读法：每部分都是”概念 → 样本与可视化（**样本页**）→ 可运行程序（**代码页**）→ 清单”的循环。

为什么整備最难 Why Preparation Dominates the Work

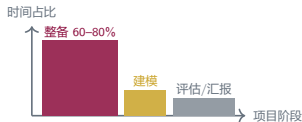
垃圾进，垃圾出

错误的输入会被模型**忠实地放大**：算法不会报错，只会给出看起来合理的错误结论。

整備不是“脏活”：它决定结论的可信度，也是项目 1 的主要工作量。

整備的三类工作

- ▶ **合并**：把多份数据对齐成一张表；
- ▶ **清洗**：修正错误、删除重复、处理异常与缺失；
- ▶ **变换**：单位、类型、分布、编码的调整。



五项质量特征 Five Characteristics of Quality

特征	含义（原课件表述）	可计算的指标
有效性 Validity	数据符合已定义的业务规则或约束的程度	正则/范围校验通过率、日期可解析率
准确性 Accuracy	数据接近真值的程度	与权威源比对、不可能值（负金额、超界）占比
完整性 Completeness	所有必需数据可知的程度	缺失率、按列/按行/成块缺失
一致性 Consistency	同一数据集内与跨数据集的一致程度	类别变体数、跨源同一实体的取值差异
单位一致 Uniformity	数据使用同一计量单位的程度	单位混入检测、量纲换算后差异

原课件把第五项写成 **Uniformity（单位一致性）**；课程大纲的表述是**唯一性 Uniqueness**。两者都重要：前者管“同一件事的单位是否统一”（73 页展开），后者管“主键是否重复”——本讲两页都讲。

Measure quality before fixing it:
numbers, not adjectives.

来源：五特征表述沿用原课件第 3 页；指标列为本讲新增的可操作口径。

从特征到动作 From Dimension to Action

特征	检测代码 (pandas)	处置动作
有效性	<code>str.match()</code> 、 <code>to_datetime(errors="coerce")</code>	修正格式、丢弃不可修复行、加校验规则
准确性	范围/业务规则断言、与权威表比对	复核原始凭证、回退到源系统、标记待核
完整性	<code>isna().mean()</code> 、缺失模式聚类	见 Part 4: 插补 / 剔除 / 免插补
一致性	<code>nunique()</code> 、别名映射表	建立映射表统一到标准编码 (46 页样例)
单位一致	单位列 + 换算函数、量纲审计	统一到基准单位并在元数据中写明
唯一性	<code>duplicated()</code> 、 <code>merge(validate=...)</code>	按业务规则去重、保留审计痕迹

原则：先量化，再决定；每一次处置都要能回答“为什么这样做、影响多少行”。

来源：检测代码与处置动作据本讲代码页 (16、42、43、53 页) 整理。

粒度 Grain: 一行代表什么 What Does One Row Mean

三个必须先问清的问题

1. 实体粒度: 一行是订单、客户, 还是门店?
2. 时间粒度: 日、月、季, 还是快照时点?
3. 聚合层级: 原始观测, 还是已汇总的合计?

	时间 (月)					
客户 A	•	•	•	•	•	•
客户 B	•	•	•	•	•	•
客户 C	•	•	•	•	•	•

一行 = 客户 × 月
(面板粒度)

粒度错位的后果

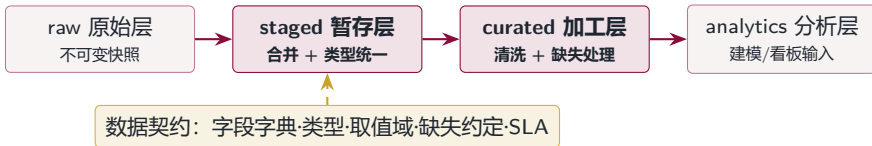
“客户-月”表当“客户”表 → 客户重复计数;
含“合计”行的表当明细 → 均值被抬高。

来源: 原课件“截面/时序/面板”定义在本讲的工程化延伸; 与 L2 数据结构一节衔接。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 数据质量与

整备总览 Quality and Preparation at a Glance

数据分层与数据契约 Layering and Data Contracts



为什么分层

出错时能回退到上一层重跑，而不是从头再抓一遍；每一层都可被断言检查，责任边界清楚。

契约写什么

字段名与含义、类型、单位、允许取值、缺失是否允许、更新频率与变更通知方式。

本讲所有程序都遵守这条线：13_01 产出 raw 层脏数据，13_05 产出 curated 层分析表。

来源：数据契约与分层是现代数据栈的通用实践（与 L2 的“数据契约与数据产品”呼应）。

清洗、转换、重塑：三件事不要混 Clean vs Transform vs Reshape

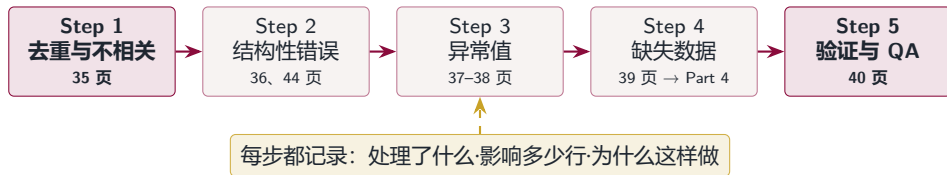
工作	定义（原课件）	典型动作与所在页
清洗 Cleaning	移除或修复“不属于这个数据集”的数据	去重、结构纠错、异常判定、缺失处理（Part 3-4）
转换 Transformation	把数据从一种格式/结构变成另一种（wrangling / munging）	类型转换、单位统一、对数与标准化（Part 6）
重塑 Reshaping	改变数据的排列形式而不改变内容	宽 ↔ 长、透视、交叉表、堆叠（Part 2）

三者经常在同一脚本里出现，但要**分开记录**：
清洗会改变事实，转换不改变事实，重塑只是
换个摆法。

Cleaning removes what does not belong;
transformation changes the form; reshaping
changes the layout.

来源：定义沿用原课件“Data cleaning vs data transformation”一页。

清洗五步法 The Five Cleaning Steps (原课件主线)



原课件的原话：**没有一套放之四海皆准的清洗步骤**——数据不同、步骤就不同；但你必须**建立模板**，才能保证每次都做对。

A template beats improvisation: same steps, logged, in the same order, every time.

来源：五步法沿用原课件第 11-15 页；“记录处理影响”为本讲新增的审计要求（42 页代码实现）。

可复现五原则 Five Rules for Reproducible Preparation

- ▶ **快照不可变**：原始文件按日期落盘，永不覆盖；
- ▶ **脚本化**：所有处理写成代码，手工改一次就无法复现；
- ▶ **断言前置**：合并前查键唯一、清洗后查值域，失败即停；
- ▶ **幂等落盘**：重跑结果一致（去重 + 覆盖写 + 固定种子）；
- ▶ **日志可审计**：每步记录行数、动作与理由。

课堂检验方式

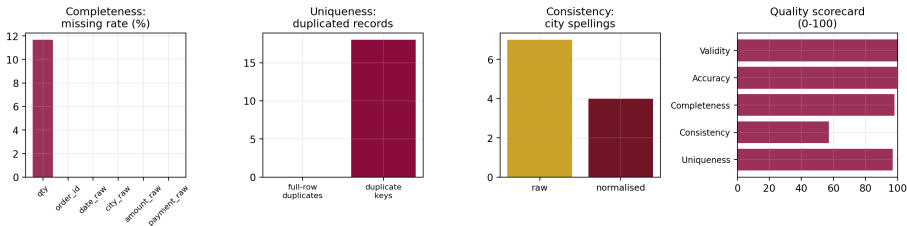
把脚本 + 原始数据交给同学，不给口头说明：能否得到与你完全相同的分析表？

本讲资源

固定种子 SEED = 20260317 · 一键重跑
python run_all.py · 清洗日志
l3_cleaning_log.csv

来源：原则据数据工程实践整理；与 L2 的“版本与可复现”、L4 的报告要求衔接。

【样本 + 可视化】质量画像仪表盘 Quality Profile



样本

13_orders_dirty.csv (600 行订单): 18 个重复主键、城市写法 7 种、支付状态 6 种写法、70 行缺数量——五维评分一次看清。

程序 13_01_quality_profile.py (逐行英文注释) · 数据 website/data/lecture03/ · 评分卡

13_quality_scorecard.csv

【代码】质量画像与断言 Profiling a Table

```
1 parsed_date = pd.to_datetime(raw["date_raw"], format="mixed", # three source formats, best effort
2                             dayfirst=True, errors="coerce") # unparsable values become NaT
3 amount = (raw["amount_raw"]
4           .str.replace(r"[\d.\-]", "", regex=True) # keep digits, dot and minus only
5           .pipe(pd.to_numeric, errors="coerce")) # unparsable leftovers become NaN
6 dup_rows = int(raw.duplicated().sum()) # fully duplicated rows (none planted)
7 dup_keys = int(raw["order_id"].duplicated().sum()) # duplicated business keys (18 planted)
8 missing_rate = raw.isna().mean().mul(100).round(2) # completeness: share missing per column
9 bad_amount = int(((amount <= 0) | (amount > 100000)).sum()) # accuracy: impossible magnitudes
10 city_raw_n = raw["city_raw"].nunique() # consistency: variants seen in the raw column
11 city_clean_n = raw["city_raw"].str.strip().str.title().nunique() # variants after normalisation
12 valid_date_rate = 100 * parsed_date.notna().mean() # validity: share of parsable dates
13 assert raw["order_id"].str.match(r"^S0\d{7}$").mean() == 1 # fail fast if ids break the pattern
```

要点：把“质量”翻译成可计算、可断言的数字——这就是质量报告的第一页。完整程序见 [website/code/lecture03/l3_01_quality_profile.py](https://github.com/zhaochen13/website/tree/main/code/lecture03/l3_01_quality_profile.py)。

小结：整备的验收标准 Acceptance Criteria

四条验收标准

1. **可复现**：换一台机器、换一个人，结果一致；
2. **可解释**：每个处理动作都有理由与影响范围；
3. **可追溯**：从分析表的任一行能追到原始快照；
4. **可验证**：有断言与质量报告，不靠肉眼检查。

进入 Part 2 之前，先自查

- ▶ 我的表**一行代表什么**？粒度写下来了吗？
- ▶ 五维质量分数各是多少？最差的一项是哪个？
- ▶ 原始数据有没有留**不可变快照**？

下一部分 (18-33 页)：把多份数据**合并与重塑**成一张可以分析的表。

Append: 纵向合并 (加观测) Adding Observations

原课件的定义

"Append adds observations to the existing variables." 追加的是**观测**；它**不要求两份数据变量完全相同**——例如已有病人数据，又收到更多病人的数据。

pandas 对应写法

```
pd.concat([q1, q2], axis=0,
          ignore_index=True)
```

列名不一致时按名字对齐，**对不上的列自动填 NaN**。

来源：定义沿用原课件 Append 一页（第 4 页）；工程动作据本讲代码页 I3_02 整理。

追加的三个必做动作

1. **对齐 schema**：改名、统一类型与单位；
2. 加 **source** 列，记录每行来自哪份文件；
3. 拼完做**行数守恒断言**：合计 = 各来源之和。

同名不同义最危险：A 的 **amount** 是元、B 的是千元，拼完看不出错，结论全错（见 73 页）。

Merge: 横向合并 (加变量) Adding Variables

原课件的定义

"Merge adds variables to the existing observations." 合并的是**变量**；它**不要求两份数据的观测完全相同**——例如已有受访者数据，又收到问卷第二部分的回答。

pandas 写法 (原课件 Dataset 1 / 2)

```
d1.merge(d2, on="person", how="inner",  
validate="1:1")
```

Stata 对照: merge 1:1 person using dataset2
多行对一行: validate="m:1" (原课件 merge m:1 person)

来源: 定义与 Dataset 1/2 例子沿用原课件第 5、31-33 页; validate/indicator 为 pandas 的显式校验手段。

三个必查问题

1. 键是哪一列，是否**唯一**？
2. 预期基数: 1:1 / 1:m / m:1 / m:m?
3. 匹配不上的行**去哪了**？看 indicator 分布。

joinby: 组内两两组合 All Pairwise Combinations within a Group

原课件的定义

"joinby is similar to merge but forms **all combinations** of the observations where it makes sense." 例如: A 表是**父母**、B 表是**子女**, joinby familyid 会生成" 每一个父母 $\times$ 他的每一个子女" 的数据集。

组内配对的行数

父母 2 人 $\times$ 子女 3 人 = 6 行; $\sum$ 所有家庭即为结果行数。用途: 构造配对数据集。

pandas 做法与风险

```
parents.merge(children, on="family_id",  
how="inner")
```

结果行数 = $\sum$ (父母数 $\times$ 子女数): **行数会被放大**, 做" 人均" 前先想清分母是谁。

来源: joinby 定义与父母/子女例子沿用原课件第 6 页; pandas 等价写法与行数放大提醒为本讲新增。

四种连接语义与 union Four Join Types and Union

连接	结果（原课件表述）	匹配不上时
Inner 内连接	只保留两张表都能匹配的值	不匹配的值被完全丢弃
Left 左连接	保留左表全部的值 + 右表匹配值	左表无匹配 → 右表字段为 null
Right 右连接	保留右表全部的值 + 左表匹配值	右表无匹配 → 左表字段为 null
Full outer 全外	保留两张表全部的值	任一侧无匹配 → 另一侧字段为 null
Union 并集	不是连接：把一张表的行追加到另一张表	要求字段数量/名称/类型尽量一致

原课件（Tableau 文档）的实务建议：如果你不确定该用哪种连接，就用 relationships——意思是不确定时先别急着连，把关系与粒度想清楚再动手。

Row counts are the fastest way to see which join you actually performed.

来源：五类语义与表述沿用原课件第 7 页（Tableau Join Types 帮助文档）。

【样本 + 可视化】五种连接的行数对照 Join Types on Real Keys

inner join: 5 rows, 0 unmatched

customer_id	order_id	amount
C001	O001	988.5
C001	O002	207.04
C002	O003	285.62
C003	O004	514.53
C003	O005	334.15

left join: 6 rows, 1 unmatched

customer_id	order_id	amount
C001	O001	988.5
C001	O002	207.04
C002	O003	285.62
C003	O004	514.53
C003	O005	334.15
C004	NULL	NULL

right join: 8 rows, 0 unmatched

customer_id	order_id	amount
C001	O001	988.5
C001	O002	207.04
C002	O003	285.62
C003	O004	514.53
C003	O005	334.15
C005	O006	102.04

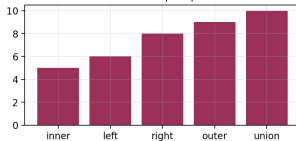
outer join: 9 rows, 1 unmatched

customer_id	order_id	amount
C001	O001	988.5
C001	O002	207.04
C002	O003	285.62
C003	O004	514.53
C003	O005	334.15
C004	NULL	NULL

union: append rows (keeps duplicates)

order_id	customer_id
O001	C001
O002	C001
O003	C002
O004	C003
O005	C003
O006	C005

rows returned per operation



左表 4 个客户、右表 8 笔订单 (含 3 笔“孤儿”订单、1 个“零订单”客户):

inner 5 行 · left 6 · right 8 · outer 9 · union 10——行数变化就是连接语义的证据。

程序 13_03_join_types_validate.py · 样本 13_customers.csv、13_orders.csv (website/data/lecture03/)

连接基数：行数为什么会爆炸 Cardinality Matters

基数	含义	pandas 写法
1:1	两侧键都唯一	<code>validate="1:1"</code>
1:m	左唯一、右可重复	<code>validate="1:m"</code>
m:1	左可重复、右唯一	<code>validate="m:1"</code>
m:m	两侧都可重复	慎用：行数 $\times$

课程示例：客户表多了一行重复的 C001，连接后 5 行变 8 行——没有 `validate` 时 pandas 只会默默放大。

让错误显式暴露

```
try: ... merge(validate="1:1")
except pd.errors.MergeError as err:
    print(err)
→ "Merge keys are not unique ... not a one-to-one merge"
```

对账 (receipt) 习惯

连接前后记录：左表行数、右表行数、结果行数、各 `indicator` 分组行数。四个数字放一起，“悄悄丢行/悄悄翻倍”无所遁形。

来源：基数概念与 `validate` 用法据 pandas 官方文档 (Merge, join, concatenate and compare)。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲 数据整备 | 合并与重塑

Combine and Reshape

连接陷阱清单 Join Pitfalls

- ▶ **重复键**：键不唯一 → 行数相乘 (23 页示例：5 行变 8 行)；
- ▶ **空键**：NULL 与任何值都不相等，含空键的行永远匹配不上；
- ▶ **类型不一致**："001" 与 1 看起来一样，却匹配失败；
- ▶ **格式噪声**：大小写、首尾空格、全半角、不可见字符；
- ▶ **键的语义漂移**：同一列在不同年份代表不同实体。

连接前跑一遍

1. 键列是否**唯一**? `duplicated().sum()`
2. 键的 `dtype` 是否一致?
3. 有没有**空键**? 占比多少?
4. 归一化: `strip().lower()`、去全角、统一编码;
5. 连完立即用 `indicator` 分组对账。

最贵的错误不是报错，而是**不报错的错**：行数悄悄变了，模型照样跑得出来。

来源：陷阱清单据数据工程实践与本讲样本 (I3_02、I3_03) 中出现的问题整理。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲:数据整备 | 合并与重塑

Combine and Reshape

键标准化与实体对齐 Key Normalisation

四类归一动作

1. **形式归一**：trim、大小写、全角转半角、统一编码；
2. **别名映射**：{" 深圳市": "Shenzhen"}；
3. **编码体系对齐**：行政区划、行业代码、股票后缀；
4. **实体消解**：改名、母子关系、重名 → 实体主表 + 映射表。

本讲样本效果

城市列 7 种写法 → 2 个城市；支付状态 6 种写法 → 4 个取值（含显式 unknown）。归一化前后的变体数，是“一致性”最直观的量化指标。

映射表要**版本化** (l3_city_names_map.csv)：它决定了你如何解释数据，属于方法的一部分。

来源：形式归一与别名映射为通用实践；本讲映射表示例见 website/data/lecture03/l3_city_names_map.csv。

【代码】append / concat 与 schema 对齐 Aligning Two Extracts

```
1 naive = pd.concat([q1, q2], ignore_index=True) # stack the two extracts as they are
2 print("naive concat shape:", naive.shape) # 500 x 9: renames create empty columns
3 print(naive.isna().sum()) # every renamed column is a block of NaN
4 RENAME = {"OrderID": "order_id", "store": "store_id", "quantity": "qty"} # canonical names
5 def align(df, source): # normalise one extract to the target schema
6     out = df.rename(columns=RENAME).copy() # rename first, then work on a copy
7     out["date"] = pd.to_datetime(out["date"], format="mixed", # mixed formats -> one dtype
8                                dayfirst=True, errors="coerce") # unparsable dates become NaT
9     out["amount"] = pd.to_numeric( # amounts still carry currency symbols
10        out["amount"].astype(str).str.replace(r"[^\d.\-]", "", regex=True), # strip noise
11        errors="coerce") # unparsable values become NaN
12     out["qty"] = pd.to_numeric(out["qty"], errors="coerce").astype("Int64") # nullable integer
13     out["channel"] = out.get("channel", pd.Series(pd.NA, index=out.index)) # fill missing column
14     out["source"] = source # provenance: which file did this row come from
15     return out[["order_id", "date", "store_id", "amount", "qty", "channel", "source"]] # one order
16 combined = pd.concat([align(q1, "Q1"), align(q2, "Q2")], ignore_index=True) # 500 x 7
17 assert len(combined) == len(q1) + len(q2), "concat lost or invented rows" # row conservation
18 assert combined["order_id"].is_unique, "duplicate order ids after concat" # key uniqueness
```

对照：朴素拼接 9 列全是空洞，对齐后 7 列干净；两条断言把“能跑”升级为“可信”。完整程序 13_02_join_concat.py。

【代码】安全合并: validate 与 indicator Safe Merges

```
1 print(dataset1.merge(dataset2, on="person", how="inner", validate="1:1")) # merge 1:1 person
2 print(dataset3.merge(dataset2, on="person", how="left", validate="m:1")) # merge m:1 person
3 print(pd.concat([dataset1, dataset4], ignore_index=True)) # append using dataset4
4 for how in ["inner", "left", "right", "outer"]: # the four join types of the lecture
5     res = customers.merge(orders, on="customer_id", how=how, # merge on the business key
6                           indicator=True) # record where every row came from
7     print(how, len(res), res["_merge"].value_counts().to_dict()) # rows and provenance mix
8 dup_keys = pd.concat([customers, customers.iloc[[0]]], ignore_index=True) # plant a duplicate key
9 try: # expect a loud failure, not a silent blow-up
10     dup_keys.merge(orders, on="customer_id", how="left", validate="1:1") # 1:1 promised
11 except pd.errors.MergeError as err: # pandas raises MergeError for broken promises
12     print("validate=1:1 correctly raised:", str(err).splitlines()[0]) # message names the problem
13 exploded = dup_keys.merge(orders, on="customer_id", how="left") # without validate: rows multiply
14 print("rows without validate:", len(exploded), "vs customers:", len(customers), len(dup_keys)) # 8 vs 5
15 safe = orders.merge(customers, on="customer_id", how="left", validate="m:1") # many -> one
```

原课件的 Stata 写法 (merge 1:1 / merge m:1 / append) 在这里逐条对应成 pandas; 区别是 pandas 可以声明基数并让违规直接报错。

宽表 ↔ 长表 Wide and Long Layouts

两种摆法

- ▶ **宽表 wide**: 一行一个实体, 时间/类别在列上;
- ▶ **长表 long / tidy**: 一行一个观测, 变量在列上。

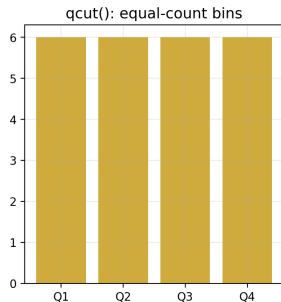
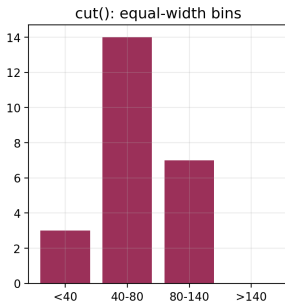
tidy data 原则: 每行一个观测、每列一个变量。建模与绘图库通常期望长表, 对外汇报常要宽表。

来源: pandas 官方文档 Reshaping and pivot tables (原课件第 37 页的主题)。

pandas 工具与往返一致性

`melt` (宽 → 长) `·pivot` (长 → 宽) `·wide_to_long` `·stack` / `·unstack`
本讲样本断言: `back_to_wide` 与 `wide` 形状相同且数值全等。

透视与交叉表 Pivot Tables and Crosstabs

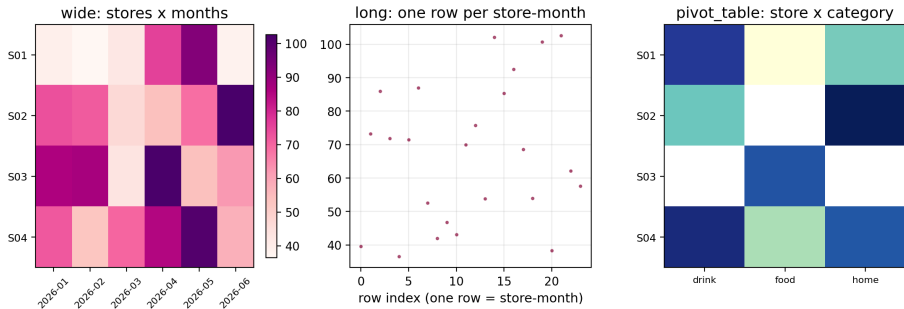


crosstab 出频数,

pivot_table 出聚合值 (默认均值); cut 按等宽分箱, qcut 按等频分箱 (本例 6/6/6/6)。

程序 13_04_reshape_toolbox.py ·分箱与交叉表是“把连续变量变成类别”的两种标准做法 (候选入模特征)

【样本 + 可视化】一份数据的三种形态 One Table, Three Layouts



4 家门店 × 6 个

月：宽表（左）适合看结构、长表（中）适合建模、透视表（右）适合汇报；样本断言验证了宽 → 长 → 宽的往返无损（4×6 形状与数值全等）。

样本 13_sales_wide.csv、13_sales_long.csv、13_sales_pivot.csv (website/data/lecture03/)

【代码】重塑工具箱 The Reshaping Toolbox

```
1 long = wide.reset_index(names="store_id").melt( # wide -> long, one row per store-month
2     id_vars="store_id", var_name="month", value_name="sales") # keep the identifier column
3 back = long.pivot(index="store_id", columns="month", values="sales") # long -> wide again
4 assert np.allclose(back.values, wide.values), "values changed in the round trip" # lossless
5 pivot = long.pivot_table(index="store_id", columns="category", # aggregate by two categories
6     values="sales", aggfunc="mean").round(1) # mean sales per store-category
7 counts = pd.crosstab(long["store_id"], long["category"]) # frequency table, not an aggregate
8 long["bucket"] = pd.cut(long["sales"], bins=[0, 40, 80, 140, 1000], # equal-width bins
9     labels=["<40", "40-80", "80-140", ">140"]) # readable labels
10 long["quartile"] = pd.qcut(long["sales"], 4) # equal-count bins: 6 rows in every quartile
11 long["cat_code"] = pd.factorize(long["category"])[0] # integer encoding of a category column
12 dummies = pd.get_dummies(long["category"], prefix="cat") # one-hot encoding for models
13 restored = pd.from_dummies(dummies).rename(columns=lambda c: "category") # invert one-hot
14 stacked = pivot.drop(columns="total").stack() # columns become a row level (MultiIndex)
15 unstacked = stacked.unstack() # and back again
16 items.explode("items") # list-like cell -> one row per item (3 -> 6 rows)
```

这 11 行覆盖了原课件“Reshaping and pivot tables” 整页的工具清单：pivot、pivot_table、stack/unstack、melt/wide_to_long、get_dummies/from_dummies、explode、crosstab、cut、factorize。

重塑陷阱 Reshaping Pitfalls

- ▶ **MultIndex 对齐错位**: `stack()` 之后索引多一层, 忘了 `reset_index()` 就会得到奇怪的表;
- ▶ **重复列名**: `pivot` 遇到同一 (index, columns) 组合有多行**直接报错**——先聚合 (`pivot_table`) 再透视;
- ▶ **类型退化**: `melt` 把 int 与 float 混在一列 → 整列变 float (金额看起来多了小数);
- ▶ **explode 产生空行**: 空列表会变成 NaN 行, 行数对不上要在断言里检查;
- ▶ **列顺序漂移**: 不同批次列顺序不同, 下游按位置取列就会错位——始终**按列名**取。

重塑不改事实, 只改摆法; 如果重塑后**数字变了**, 一定是某一步错了。

三道防线

1. **形状断言**: 重塑前后行数 $\times$ 列数的关系是否可解释;
2. **数值断言**: 往返转换后 `np.allclose` 校验;
3. **键断言**: 重塑后 (index, columns) 组合是否唯一。

来源: 陷阱据 pandas 重塑文档与本讲样本实现整理。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 合并与重塑

Combine and Reshape

小结：合并与重塑验证清单 Reconciliation Checklist

合并 (merge)

- ▶ 键是否唯一、类型是否一致、空键占比；
- ▶ 基数声明 (validate) 与实际是否吻合；
- ▶ indicator 分组行数是否与预期一致；
- ▶ 结果行数 = 两表匹配行 + 保留的未匹配行。

追加 (concat) 与重塑

- ▶ 列名/类型/单位是否先对齐；
- ▶ 行数守恒：合计 = 各来源之和；
- ▶ 往返无损：宽 $\leftrightarrow$ 长、stack $\leftrightarrow$ unstack；
- ▶ source 列与 filled_flag 等标记列是否保留。

下一部分 (34–47 页)：清洗五步法——重复、结构错误、异常值、缺失、验证。

清洗是什么，收益与代价 What Cleaning Is Worth

原课件的定义

数据清洗是修复或移除错误、损坏、格式错乱、重复、不完整数据的过程。多源合并时最容易出现重复与错标：如果数据是错的，结论和算法都不可靠——即使它们看起来是对的。

原课件同时强调：没有一套固定的清洗步骤，但必须为你的数据建立模板，保证每次都“做对”。

收益

减少多源错误·提高决策信息质量·让口径可解释·让错误可定位与可复盘

代价（本讲补充）

清洗会改变事实：删行是丢信息，填值是造信息。所以必须记录改了什么、多少行、为什么。

来源：定义与收益沿用原课件第 8-9 页；“清洗的代价”为工程实践补充。

Step 1: 去重与不相关观测 Remove Duplicates and Irrelevant Rows

原课件的要点

- ▶ 重复最常发生在数据收集阶段：合并多源、抓取网页、从多个部门取数时；
- ▶ 去重 (de-duplication) 是清洗中占比最大的一块；
- ▶ 不相关观测：不符合研究问题的行（例如研究千禧一代却混入了其他世代）。

三种“重复”要分清

1. 完全重复：所有字段都一样 → 直接删；
2. 主键重复：同一订单出现两次（可能金额不同）→ 按业务规则保留哪一条要写清楚；
3. 业务重复：同客户同日同金额 → 可能是两笔真实订单，不能盲删。

本讲样本：600 行里有 18 个重复主键，清洗日志记录“删了多少、依据是什么”。

来源：要点沿用原课件第 11 页；三类重复的区分与处理规则为工程实践补充。

Step 2: 修复结构性错误 Fix Structural Errors

原课件的例子

度量或传输数据时出现奇怪的命名、拼写错误、大小写不一致，会导致类别被错误地拆成多个：数据里同时出现 "N/A" 与 "Not Applicable"，它们本应作为同一个类别分析。

常见结构错误

首尾空格·全半角混用·单位混入 ("1.2 thousand CNY")·日期格式三种并存·布尔值写成 yes/Y/1

处理套路 (三步)

1. 先**盘点**：value_counts() 看变体数；
2. 再**建映射表**：raw → canonical (可版本化、可复现)；
3. 最后**断言**：归一化后类别数应等于预期值。

本讲样本：城市 7 种写法 → 2 个城市；支付状态 6 种写法 → 4 个取值。

来源：要点与原例沿用原课件第 12 页；三步套路为可复现实践补充。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲·数据整备 | 数据清洗

Data Cleaning

Step 3: 异常值该删还是留 Outliers: Remove or Keep

原课件的立场

出现一次性的“看起来不合群”的观测时：如果它有**正当理由**（如录入错误），删除有助于数据表现；但有时**正是异常值证明了你的理论**。**离群不等于错误**——这一步的目的是判断该值是否有效。

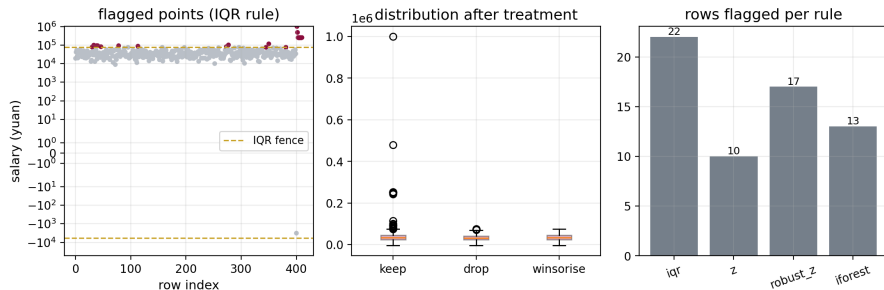
决策顺序：1) 是不是错误？（回到原始凭证）2) 是不是真实事件？（记录）3) 与研究问题是否相关？——只有“错误且无法修复”才删；“真实但极端”应保留并在**报告中说明**。

来源：立场与原例沿用原课件第 13 页；三种处置与其对均值/方差的影响见 38 页与代码页 43 页。

三种处置（本讲补充）

- ▶ **保留**：真实极端值（如并购公告日的成交量）；
- ▶ **缩尾 winsorise**：保留行但压到分位边界；
- ▶ **删除**：确认是录入/传输错误，并记录条数与理由。

异常值判定三法 Three Detection Rules



IQR (箱线规则) 标出

22 行、Z-score 10 行、MAD 稳健 Z 17 行、IsolationForest 13 行；四法一致的只有 10 行——说明“异常”依赖方法与阈值。

程序 13_06_outliers_iqr_z_iforest.py 样本 13_income_outliers.csv (含 2 个录入错误、1 个真实极端值、8 行特殊群体)

Step 4: 处理缺失 (引向 Part 4) Handle Missing Data

原课件的三个选项

1. 删除含缺失的观测——会丢信息，删前要清楚代价；
2. 依据其他观测填补——可能引入假设，损失数据完整性；
3. 改变数据用法，绕过缺失——用对缺失不敏感的方法。

原课件提醒：没有一种是最优的，都要权衡。

来源：三选项沿用原课件第 14 页；“先问机制”的流程与 Part 4 衔接。

本讲补充：先回答两个问题

- ▶ 缺失多少？(按列、按行、成块)
- ▶ 缺失为什么发生？(机制：MCAR / MAR / MNAR)

只有先弄清机制，才能判断“删除是否引入偏差”。

下一部分 (48–60 页) 专门讲缺失：机制、三种策略、插补方法与代价。

Step 5: 验证与 QA Validate and QA

原课件的五个问题

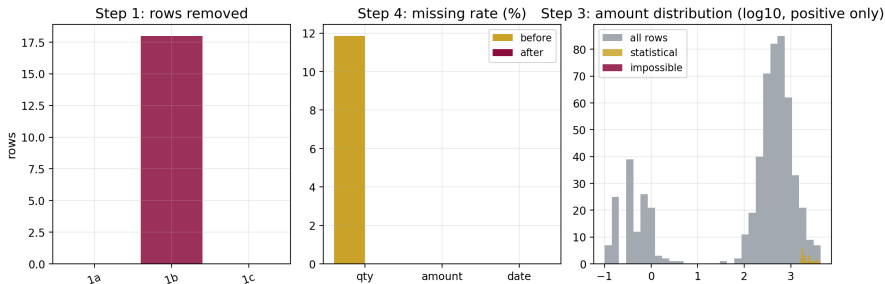
1. 数据**讲得通**吗？
2. 数据符合该字段的**规则**吗？
3. 它**支持还是推翻**了你的假设？
4. 能否从中看到**趋势**以形成下一个假设？
5. 如果不能，是**数据质量问题**导致的吗？

可执行的 QA（本讲补充）

- ▶ **断言**：类型、唯一性、值域、类别闭集，失败即停；
- ▶ **对账**：清洗前后行数与分布对照（41 页图）；
- ▶ **抽样复核**：随机抽 30 行回原始凭证核对；
- ▶ **质量报告**：结论写成“发现 + 影响 + 处理”（86 页模板）。

来源：五问题沿用原课件第 15 页；可执行 QA 的四项动作据本讲代码页与新增加模板整理。

【样本 + 可视化】清洗前后对照 Before and After Cleaning



600 行 → 582 行 (去重与不相关); 数量缺失 70 行 → 0 (中位数 6 填 + 缺失指示列); 金额分布保持右偏, 但不可能值被单独标记而不是悄悄删除。

程序 13_05_cleaning_pipeline.py · 审计日志 13_cleaning_log.csv · 结果表 13_orders_clean.csv

【代码】清洗五步法全流程 The Five Steps in Code

```
1 raw = pd.read_csv(DAT / "l3_orders_dirty.csv", dtype=str) # read the dirty sample as text
2 raw["qty"] = pd.to_numeric(raw["qty"], errors="coerce") # quantity comes back as float
3 df = raw.drop_duplicates().copy() # Step 1a: drop fully duplicated rows
4 df = df.drop_duplicates(subset="order_id", keep="first") # Step 1b: one row per business key
5 df = df.loc[~df["order_id"].isna()].copy() # Step 1c: drop irrelevant rows (no key)
6 df["city"] = df["city_raw"].map(normalise_city) # Step 2: canonical city names via alias map
7 df["payment"] = df["payment_raw"].str.strip().str.lower().map(pay_map).fillna("unknown") # Step 2
8 df["date"] = pd.to_datetime(df["date_raw"], format="mixed", dayfirst=True, errors="coerce") # Step 2 dates
9 impossible = (df["amount"] <= 0) | (df["amount"] > 100000) # Step 3: physically impossible
10 q1, q3 = df["amount"].quantile(.25), df["amount"].quantile(.75) # quartiles for the IQR rule
11 iqr = q3 - q1 # inter-quartile range
12 statistical = (df["amount"] < q1 - 1.5 * iqr) | (df["amount"] > q3 + 1.5 * iqr) # Tukey fence
13 df["flag_impossible"] = impossible # keep the flag, never delete silently
14 df["flag_outlier"] = statistical & ~impossible # statistical but plausible values
15 df["qty_missing"] = df["qty"].isna().astype(int) # Step 4: missingness is itself information
16 df["qty"] = df["qty"].fillna(df["qty"].median()) # fill with the median, documented
17 assert df["order_id"].is_unique, "order_id must be unique after cleaning" # Step 5 assertion
18 assert df["date"].notna().all(), "unparsable dates must be resolved" # Step 5 assertion
19 record("Step 4 missing handling", len(df), len(df), "qty filled with median") # audit log entry
```

每一步都写进**审计日志** (l3_cleaning_log.csv): 步骤名、前后行数、移除了多少、为什么。完整程序 l3_05_cleaning_pipeline.py。

【代码】异常值三法与处置 Three Rules, Three Treatments

```
1 core = RNG.lognormal(mean=10.4, sigma=0.45, size=400) # normal salaries in yuan
2 errors = np.array([-3200.0, 999999.0]) # two data-entry errors
3 extreme = np.array([480000.0]) # one legitimate but extreme salary
4 group = RNG.normal(loc=250000, scale=4000, size=8) # a small cluster of unusual records
5 df = pd.DataFrame({"salary": np.round(np.concatenate([core, errors, extreme, group]), 2)}) # one column
6 q1, q3 = df["salary"].quantile([.25, .75]) # quartiles of the observed column
7 iqr = q3 - q1 # inter-quartile range
8 df["iqr_outlier"] = (df["salary"] < q1 - 1.5 * iqr) | (df["salary"] > q3 + 1.5 * iqr) # Tukey fence
9 z = (df["salary"] - df["salary"].mean()) / df["salary"].std(ddof=0) # classic Z-score
10 df["z_outlier"] = z.abs() > 3 # the |z| > 3 rule
11 mad = (df["salary"] - df["salary"].median()).abs().median() # median absolute deviation
12 robust_z = 0.6745 * (df["salary"] - df["salary"].median()) / mad # robust Z for skewed data
13 iso = IsolationForest(contamination=0.03, random_state=42) # expect about 3% anomalies
14 df["iforest_outlier"] = iso.fit_predict(df[["salary"]]) == -1 # -1 marks an anomaly
15 drop = df.loc[~df["iqr_outlier"], "salary"] # treatment A: drop the flagged rows
16 wins = df["salary"].clip(lower=q1 - 1.5 * iqr, upper=q3 + 1.5 * iqr) # treatment B: winsorise
17 print(summary) # compare n, mean, median and sd side by side
```

关键结论: Z-score 对极端值过敏 (均值与标准差都被污染), MAD 稳健 Z 更适合右偏数据; 三法一致的行才最可疑。完整程序 13_06_outliers_iqr_z_iforest.py。

文本类脏数据的清洗 Cleaning Messy Strings

常见脏法

- ▶ 首尾空白与不可见字符 (`\u200b`、`\n`);
- ▶ 全角/半角混用 (1 2 0 vs 120);
- ▶ 数字中混入货币符号与千分位 (¥1,234.56);
- ▶ 单位写在值里 (1.2 thousand CNY);
- ▶ 占位符不统一 (N/A、-、null、无)。

处理顺序

1. 统一为 `str`, 去不可见字符;
2. `str.strip()`、全角转半角;
3. 用正则抽出数字部分再转数值;
4. 单位单独成列 (值 $\times$ 单位 = 基准单位);
5. 占位符统一映射为 `NA` 并记录。

本讲样本的 `amount` 列同时混了四种写法 (¥ 前缀、RMB 后缀、thousand CNY、纯数字) ——统一后金额才可比较。

来源: 脏数据类型与处理顺序据数据清洗实践整理; 样本见 I3_01 生成的 I3_orders_dirty.csv。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 数据清洗

Data Cleaning

近重复与模糊匹配 Near Duplicates and Record Linkage

为什么不能只用完全匹配

"深圳大学" 与 "深圳大學"、"Shenzhen Univ." 与 "Shenzhen University", 在计算机看来是**不同**的字符串, 在业务上看是同一个实体。

常用相似度

编辑距离 (Levenshtein) · Jaro-Winkler · 标准化后哈希 (严格相等) · 词袋/嵌入相似度

落地的三步

1. **标准化**: 去空格标点、繁转简、大小写统一;
2. **分块 (blocking)**: 只比较同省/同前缀的记录, 避免 $O(n^2)$;
3. **人工确认**: 相似度阈值以上抽样复核, 确认后写入**映射表**并版本化。

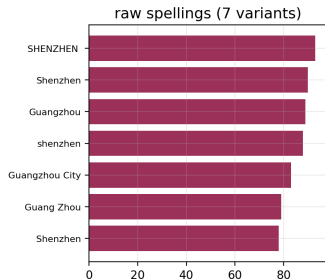
原则: 能靠规则确定的不要靠猜; 必须靠猜的, 要留下置信度与人工确认记录。

来源: 相似度与 blocking 为 record linkage 通用做法; 本讲用标准库 difflib 即可演示 (见代码页 42 页的映射表思路)。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 数据清洗

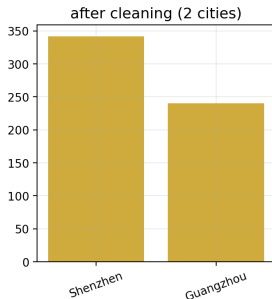
Data Cleaning

【样本 + 可视化】结构错误与类别规范化 Structural Repair



alias mapping applied

raw	canonical
shenzhen	Shenzhen
shēnzhèn	Shenzhen
guangzhou city	Guangzhou
guang zhou	Guangzhou



左：原始 7 种城市写法；中：别名映射表 (raw $\rightarrow$ canonical)；右：清洗后只剩 2 个城市。映射表本身也是交付物，它决定了你如何解释数据。

样本 `13_city_names_raw.csv`、`13_city_names_map.csv` (website/data/lecture03/)

小结：清洗的边界与审计 Cleaning Boundaries and Audit

不要清洗掉信号

异常可能是真实事件 (并购、政策、疫情); 把“看起来奇怪”当成“错误”，会让研究丢掉最有信息量的观测。先判有效，再决定去留。

清洗日志要写什么

步骤·前后行数·移除/修改条数·依据 (规则/映射表/人工确认)·责任人

三条自查

1. 我删的每一行都能说出理由吗？
2. 我填的每一个值都标注了吗 (filled_flag)?
3. 别人拿到我的脚本能复现同样的表吗？

下一部分 (48–60 页)：缺失数据——机制、策略、插补方法与代价。

缺失的形态与来源 Where Missingness Comes From

原课件的定义

缺失值指某个观测的某个变量没有存下任何值。缺失很常见，并会显著影响能从数据中得出的结论。

两个主要来源

- ▶ **无响应 nonresponse**: 某个题目或整个受访单元没有提供信息; **敏感题** (收入、健康) 更容易无响应;
- ▶ **流失 attrition**: 纵向研究中, 参与者中途退出, 导致后续测量缺失。

缺失会**降低样本代表性**, 从而扭曲对总体的推断——所以“缺多少”与“为什么缺”必须一起报。

来源: 定义、无响应与流失沿用原课件第 16 页。

MCAR: 完全随机缺失 Missing Completely at Random

定义 (原课件)

导致某个数据项缺失的事件，既独立于可观测变量，也独立于不可观测的参数，完全随机发生。

后果

在 MCAR 下，基于可用数据的分析是无偏的——但原课件提醒：现实中很少真正 MCAR。

来源：定义与后果沿用原课件第 17 页；数值为本讲 I3_07 模拟（固定种子）的结果。

本讲模拟结果

随机删掉 26.9% 的收入后，可用样本均值 16231.93 vs 真值 16265.89：偏差 -0.2%，几乎可忽略。

判断线索：缺失与任何变量都不相关 → 更像 MCAR；但仍需用 Little's MCAR 检验等方法做初步判断。

MAR: 随机缺失 Missing at Random

定义 (原课件)

缺失不是随机的，但缺失性可以被完全观测到的变量解释。

为什么可以救

原课件指出：MAR 是一个无法用统计检验证实的假设，只能依赖其实质合理性；但如果用完全信息极大似然 (FIML) 之类的方法估计参数，MAR 下可得到渐近无偏的估计。

来源：定义与 FIML 说明沿用原课件第 17 页；数值为本讲 I3_07 模拟结果。

本讲模拟结果

让“学历越高越不愿报告收入”：缺失率 27.4%，可用样本均值偏差 -10.0% ——比 MCAR 严重得多，但学历是完整变量，模型可修正。

实操：MAR 下可用“完整变量 + 回归/多重插补”修正；记得在报告中说明假设。

MNAR: 非随机缺失 Missing Not at Random

定义 (原课件)

既不是 MAR 也不是 MCAR: **缺失的值本身就与缺失的原因相关**。原课件的例子: 男性因为自己的抑郁程度而**不愿填写**抑郁量表。

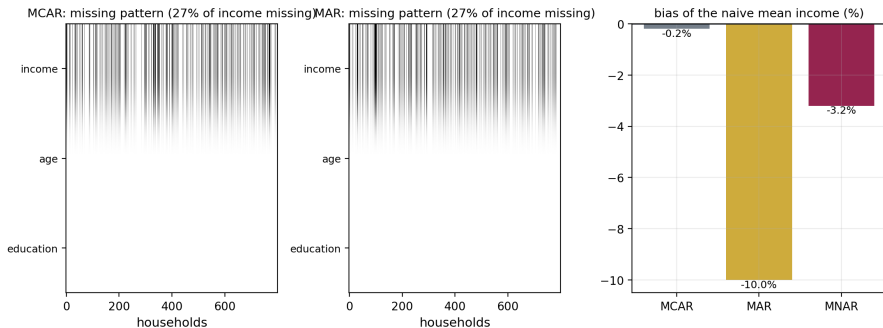
本讲模拟结果

让”收入越高越不愿回答”: 缺失 15.1%, 可用样本均值偏差 **-3.2%** ——注意: 偏差大小既取决于**机制**, 也取决于**缺失比例**。

MNAR **不可检验**: 统计上无法从数据本身证明。唯一诚实的做法是 **做敏感性分析** (在不同 MNAR 假设下看结论是否改变), 并在报告中写明假设与边界。

来源: 定义与原例沿用原课件第 17 页; 数值为本讲 I3_07 模拟结果。

【样本 + 可视化】缺失矩阵与偏差 Missingness Matrix and Bias



左两张：MCAR（随

机黑点）与 MAR（黑点集中在高学历段）的缺失矩阵；右：三种机制下朴素均值的相对偏差 -0.2% / -10.0% / -3.2%。缺失模式在图上就能看出机制。

程序 `l3_07_missing_mechanisms.py` · 偏差表 `l3_missing_bias.csv` · 三份损坏样本
`l3_income_missing_{MCAR,MAR,MNAR}.csv`

【代码】缺失机制模拟 Simulating MCAR / MAR / MNAR

```
1 mu = np.array([base[e] for e in education]) + 0.012 * (age - 22) # income grows with age
2 income = RNG.lognormal(mean=mu, sigma=0.30) # complete, known monthly incomes in yuan
3 true_mean = income.mean() # the estimand we want to recover
4 mcar_mask = RNG.random(N) < 0.30 # MCAR: every row has the same 30% chance
5 mar_p = np.where(education == "master", 0.55, # MAR: missingness depends on education,
6                 np.where(education == "bachelor", 0.30, 0.12)) # a fully observed variable
7 mar_mask = RNG.random(N) < mar_p # draw the MAR deletion pattern
8 mnar_p = np.clip((income - income.min()) / (income.max() - income.min()), 0, 1) ** 2 # richer = higher p
9 mnar_mask = RNG.random(N) < 0.15 + 0.55 * mnar_p # MNAR: high earners refuse to report
10 for name, mask in {"MCAR": mcar_mask, "MAR": mar_mask, "MNAR": mnar_mask}.items(): # three versions
11     damaged = complete.copy() # keep the complete rows untouched
12     damaged.loc[mask, "income"] = np.nan # delete the income values only
13     naive = damaged["income"].mean() # estimate the mean on the observed rows
14     bias = naive - true_mean # bias of the naive estimate
15     print(name, round(100 * bias / true_mean, 1), "%") # -0.2% / -10.0% / -3.2%
16 grid.loc[drop_firm | drop_tail, "revenue"] = np.nan # panel sample: a whole block disappears
17 coverage = grid.groupby("firm_id")["revenue"].apply(lambda s: s.notna().sum()) # coverage
```

同一份完整数据、三种缺失机制：只有 MCAR 下朴素估计才无偏。完整程序 13_07_missing_mechanisms.py。

三种处理策略 Three Strategies (原课件)

策略	原课件表述	本讲落地要点
插补 Imputation	用“填入”的值替代缺失数据	必须先写清方法与理由；插补值要 打标记
剔除 Omission	把含无效数据的样本从后续分析中丢弃	要报告 丢了多少、丢了谁 (是否系统性丢某类个体)
分析 Analysis	直接使用 不受缺失影响 的方法	例如部分重叠样本 t 检验 (原课件例子, 正态 + MCAR 下有效)

原课件提到：系统性综述曾归纳出缺失数据预防与处理的**10 项标准**，覆盖研究设计、实施、分析与报告——也就是说，“怎么处理缺失”在写研究设计时就该定下来。

来源：策略划分与原课件第 18 页一致；落地要点为本讲补充。

插补方法族 The Imputation Family

从简单到复杂

1. 均值/中位数：最快，但会压缩方差、削弱相关性；
2. 回归插补：用其他变量预测，保留关系但高估确定性；
3. KNN 插补：用相似的邻居，适合数值型、需要标准化；
4. EM 算法（原课件）：迭代估计”若有完整数据时会得到的统计量”，通常不逐条填值；
5. 多重插补 MI / MICE：生成多套数据分别分析再合并，能反映插补的不确定性。

EM 与 MI 的区别（原课件）

多重插补的每套数据都要分别分析，有时还要用较复杂的方式合并统计量；EM 则是在参数层处理缺失，而不是逐项填值。

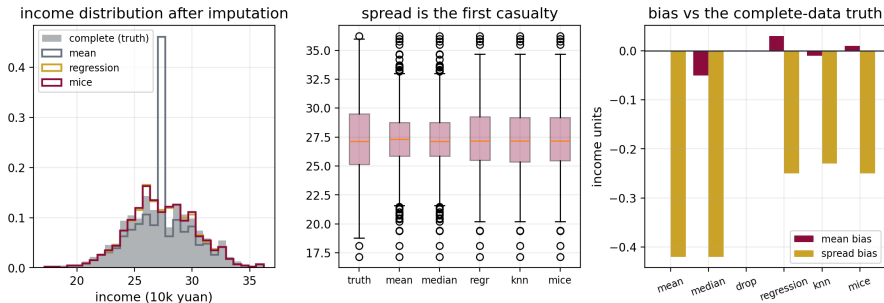
选择顺序：能用完整数据/免插补方法就不用插补；必须插补时，优先多重插补并报告敏感性。

来源：方法族与原课件第 19 页（Imputation、EM、interpolation）对应；评估口径见代码页 56 页。

【代码】五种插补方法的对比 Scoring Imputations

```
1 missing_idx = damaged["income"].isna() # rows whose income must be filled
2 known = damaged.dropna(subset=["income"]) # rows used to fit the model
3 features = ["hours", "edu", "age"] # observed predictors
4 filled = {"mean": damaged["income"].fillna(damaged["income"].mean()), # mean imputation
5          "median": damaged["income"].fillna(damaged["income"].median())} # median imputation
6 reg = LinearRegression().fit(known[features], known["income"]) # regression imputation
7 filled["reg"] = damaged["income"].copy() # start from the observed column
8 filled["reg"].loc[missing_idx] = reg.predict(damaged.loc[missing_idx, features]) # fill
9 knn = KNNImputer(n_neighbors=5).fit_transform(damaged[features + ["income"]]) # KNN
10 filled["knn"] = pd.Series(knn[:, -1], index=damaged.index) # neighbours' values
11 mice = IterativeImputer(max_iter=10, random_state=42).fit_transform(damaged[features + ["income"]]) # MICE
12 filled["mice"] = pd.Series(mice[:, -1], index=damaged.index) # MICE: chained equations
13 for name, series in filled.items(): # score every strategy against the truth
14     print(name, series.mean() - truth_mean, series.std() - truth_sd) # level / spread bias
```

【样本 + 可视化】插补前后：分布与关系 Imputation Scored



左：均值插补把分布压成一根尖峰；回归/KNN/MICE 更接近真分布。中：箱线显示 方差收缩。右：相对完整数据真值的水平与离散偏差。

程序 13_08_imputation_compare.py 指标表 13_imputation_metrics.csv 损坏样本 13_income_missing.csv

插补的代价与风险 What Imputation Costs

四个典型副作用

- ▶ **方差收缩**：填进去的值彼此太像，标准误被低估；
- ▶ **关系被制造**：回归插补会让 X 与 Y 的相关比真实更强；
- ▶ **虚假精度**：插补值被当作观测值参与后续计算；
- ▶ **假设隐身**：MAR 假设没写进报告，读者以为数据完整。

四条纪律

1. 插补值**打标记** (filled_flag)，可随时剔除重算；
2. 报告**缺失率 + 机制假设 + 方法**；
3. 做**敏感性分析**：换一种方法/假设，结论是否改变；
4. 优先**多重插补**，让不确定性进入标准误。

判断标准不是“插得像不像”，而是**下游结论是否稳健**。

来源：副作用与纪律据缺失数据文献与工程实践整理。

缺失指示变量

“没填收入”本身可能就是变量：加一列 `income_missing`，让模型同时使用**值与是否缺失**的信息（MNAR 下的实用做法）。

时间序列的插值（原课件 Stata `ipolate`）

线性插值 `ipolate y x, gen(y1);`
带外推 `..., epolate`。pandas 对照：
`s.interpolate()`、`limit_direction="both"`、`ffill/bfill`。

插值 vs 外推

- ▶ **插值 (interpolation)**: 在已知点**之间**构造新点，通常合理；
- ▶ **外推 (extrapolation)**: 在已知范围**之外**延伸，风险高，必须标注；
- ▶ 时间序列插值要**按个体分组**（面板里不能跨个体插值）。

来源：插值与外推沿用原课件第 19、27 页（Stata `ipolate/epolate`）；指示变量与分组提醒为本讲补充。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲 数据整备 | 缺失数据

Missing Data

小结：缺失处理决策树 A Decision Tree

缺失比例？按列/按行/成块？

机制？MCAR / MAR / MNAR

选择策略并记录

然后：插补值打标记 → 敏感性分析 → 写进质量报告

判断顺序

1) 先看**比例与模式** (52 页矩阵) → 2) 再判**机制** (49–51 页) → 3) 选**策略** (54 页) → 4) 用**四把尺子**评估 (56 页)

三条底线

不隐瞒缺失·不把插补值当观测·不省略机制假设

下一部分 (61–70 页)：**面板与平衡性**——当“缺失”发生在个体 × 时间格上时。

平衡面板与非平衡面板 Balanced vs Unbalanced Panels

原课件的定义

如果每个截面观测都有相同的时间期 $t = 1, \dots, T$, 面板称为平衡面板; 对非平衡面板, 时间维度记为 T_i , 因个体而异。

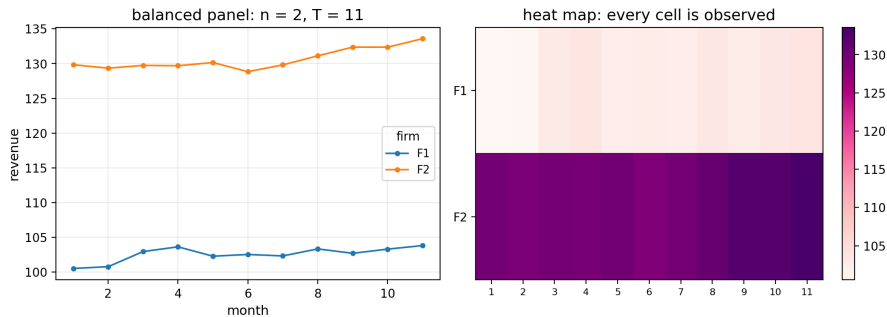
非平衡的三个常见成因: 个体中途进入 (新公司上市)、中途退出 (倒闭、退市)、部分期间未观测 (数据缺失)。

来源: 定义沿用原课件第 20 页 (Section 2 Baseline Definitions)。

为什么重要

估计方法 (固定效应、一阶差分)、滞后项构造、缺失模式都依赖面板结构; 结构没诊断清楚, 后面的模型假设就无从谈起。

【样本 + 可视化】平衡面板 $T = 11, n = 2$ A Balanced Panel



左：两家公司 11 个月的营收路径（随机游走 + 各自的水平与趋势）；右：个体 $\times$ 时间热力图，每一格都有观测——这就是平衡面板（对应原课件“Balanced Panel with $T=11$ and $n=2$ ”）。

程序 `13_09_panel_balance.py` · 样本 `13_panel_balanced.csv` (website/data/lecture03/)

名义平衡与内含缺失 Nominal Balance, Real Gaps

原课件的两张图

- ▶ "Balanced panel with $T = 11$ and $n = 2$ with missing values": 期数齐全, 但取值缺失;
- ▶ " $T_1 = 9, T_2 = 11, T_3 = 8$ and $n = 3$ ": 期数本身就不一样 (非平衡或"名义平衡")。

本讲样本的对应

- ▶ 结构: T 取 9/11/8, $n = 3$;
- ▶ 取值: F2 的第 4、5 月被人为置空 (块状缺失);
- ▶ 覆盖表: 每个个体的"观测月数 / 应有月数" = 覆盖率。

关键区分: 行是否存在 (结构) 与值是否缺失 (取值) 是两件事, 处理方式也不同: 前者决定时间维度, 后者属于缺失数据处理。

来源: 两图对应原课件第 22-23 页; 覆盖率的量化口径为本讲补充。

非平衡面板的机制与后果 Why Panels Become Unbalanced

原课件的 Remark

"While the mechanics of the unbalanced case are similar to the balanced case, a careful treatment of the unbalanced case requires a formal description of **why** the panel may be unbalanced, and the sample selection issues can be somewhat subtle."

原课件由此指向两个问题：**样本选择与流失 (attrition)**。

后果

- ▶ 幸存者偏差：留下来的个体不是随机子集；
- ▶ 时间维度不齐 → 滞后/差分样本量逐期变化；
- ▶ 缺失与结果相关时（MNAR 面板版），估计有偏。

所以：**先写清“为什么非平衡”，再决定“要不要补齐”**——盲目补齐会掩盖样本选择问题。

来源：Remark 与原话沿用原课件第 24 页。

诊断面板结构 Diagnosing the Panel: Stata xtset vs pandas

```
1 use gymdata, clear           # Stata: load the example panel
2 list id month wt if id==1 | id==2, sepby(id) # inspect two
                                     individuals
3 xtset id month                # declare panel and time variables
```

Panel variable: id (unbalanced)

Time variable: month, 1 to 12, but with gaps

```
1 df = df.sort_values(["id", "month"]) # pandas: same ordering
                                     idea
2 df.groupby("id")["month"].agg(["count", "min", "max"]) #
                                     coverage
3 df.groupby("id")["month"].apply(lambda s: s.diff().max()) #
                                     gaps
4 if df.duplicated(["id", "month"]).any(): # key must be unique
5     raise ValueError("duplicate id-month rows") # the key of a
                                     panel must be unique
```

Stata 用 xtset 一句话宣告结构并自动报告“unbalanced / with gaps”；pandas 需要自己写覆盖表 + 缺口检查 + 键唯一性断言——这正是代码页 67 页做的事。

来源：Stata 命令沿用原课件第 25 页；pandas 等价检查为本讲补充。

面板清洗与插值: Stata 与 pandas 对照 Cleaning and Interpolating

Stata (原课件第 26–27 页)

pandas / Python 等价写法

<code>drop if id==3</code>	<code>df = df[df["id"] != 3]</code>
<code>keep if year>=2015</code>	<code>df = df[df["year"] >= 2015]</code>
<code>gen X = wt*2</code>	<code>df["X"] = df["wt"] * 2</code>
<code>replace X = . if X<0</code>	<code>df.loc[df["X"] < 0, "X"] = np.nan</code>
<code>order X, before(wt)</code>	<code>df = df.reindex(columns=[...])</code>
<code>label var X "weight"</code>	列名/字段字典 (数据契约的一部分)
<code>by id: ipolate X t, gen(X_modify)</code>	<code>df.groupby("id")["X"].transform(lambda s: s.interpolate())</code>
<code>... , epolate</code>	<code>interpolate(limit_direction="both")</code> 并标注外推值

原课件强调 `ipolate` 的线性插值与 `epolate` 的外推; 工程上再补一条: 按个体分组插值, 并在结果里加 `filled_flag`。

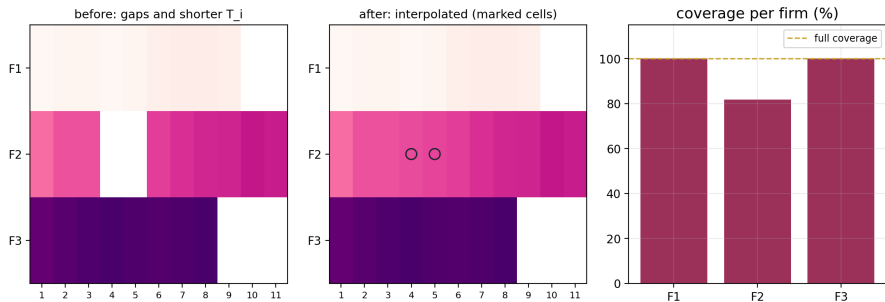
来源: Stata 命令清单沿用原课件第 26–30 页; pandas 对照为本讲补充。

【代码】面板体检与插值补齐 Panel Health Check

```
1 spec = {"F1": 9, "F2": 11, "F3": 8}      # T1=9, T2=11, T3=8, n=3 (unbalanced by design)
2 unbalanced.loc[(unbalanced["firm_id"] == "F2") & # plant a block of missing months
3               (unbalanced["month"].isin([4, 5])), "revenue"] = np.nan # F2 loses months 4-5
4 coverage = (unbalanced.assign(observed=unbalanced["revenue"].notna()) # per firm coverage
5             .groupby("firm_id")["observed"].agg(periods="count", observed="sum")) # per-firm counts
6 coverage["T_i"] = [spec[f] for f in coverage.index] # declared horizon per firm
7 coverage["coverage_pct"] = (100 * coverage["observed"] / coverage["periods"]).round(1) # coverage %
8 coverage["balanced"] = coverage["periods"] == coverage["periods"].max() # same T for all?
9 repaired = unbalanced.sort_values(["firm_id", "month"]).copy() # sort before interpolating
10 repaired["revenue_interpolated"] = (repaired.groupby("firm_id")["revenue"] # within each firm
11                                   .transform(lambda s: s.interpolate(limit_direction="both")) # fill
12 repaired["filled_flag"] = repaired["revenue"].isna() & repaired["revenue_interpolated"].notna() # mark
13 repaired["revenue"] = repaired["revenue_interpolated"] # promote the filled values
14 wide = repaired.pivot(index="firm_id", columns="month", values="revenue") # matrix after repair
15 print("cells still missing:", int(wide.isna().sum().sum())) # 5: interpolation cannot invent periods
16 print("imputed cells:", int(repaired["filled_flag"].sum())) # 2 cells were manufactured
```

教学要点：插值只能填**已存在的行**；**根本不存在的期间**（F1 的第 10–11 月、F3 的第 9–11 月）不会被补出来——要补就必须**显式决定**（补齐空行 + 标注来源）。

【样本 + 可视化】补齐前后的面板热力图 Panel Before and After



左：缺口与更短的 T_i (白格 = 无观测)；中：按个体线性插值后，**空心圈标出被插补的格子**；右：每个个体的覆盖率 (T_i 不同，覆盖率也不同)。

程序 13_09_panel_balance.py · 样本 13_panel_unbalanced.csv、13_panel_repaired.csv、覆盖表
13_panel_coverage.csv

案例：中国地级市人口数据 China's Prefectural Population Panel

原课件的案例

用中国地级市人口数据演示面板整备的全过程（原课件第 28 页“An Example”）。

真实世界会遇到什么

- ▶ **行政区划变更**：撤县设区、地市合并、改名 → 个体定义变了；
- ▶ **口径变更**：常住人口 vs 户籍人口，统计年鉴改版；
- ▶ **覆盖不全**：部分年份/部分城市缺数（尤其早期年份）。

处理顺序

1. 建立**城市主表**（含历史沿革与代码映射）；
2. 逐年对齐（用**行政区划代码**而不是城市名）；
3. 标注**口径变更年份**，必要时分段建模；
4. 覆盖率表 + 缺失模式图，随数据一起交付。

来源：案例沿用原课件第 28 页；口径与行政区划处理要点数据统计年鉴使用经验整理，与 L2 数据源一节衔接。

小结：面板整備清单 Panel Preparation Checklist

结构

- ▶ 主键 (id, time) 是否**唯一**？
- ▶ 时间轴是否连续？缺口在哪、多大？
- ▶ 每个个体的 T_i 是多少？**平衡还是非平衡**？
- ▶ 个体进入/退出样本的**原因**是什么？

取值与留痕

- ▶ 缺失是“行不存在”还是“值缺失”？
- ▶ 插补/补齐的格子是否**打标记**？
- ▶ 覆盖率表与缺失矩阵是否随数据交付？
- ▶ 平衡性声明是否写进报告？

下一部分 (71–83 页)：**变换、工具与工程化**——单位、分布、编码，以及把整備做成可监控的管道。

变量变换 Variable Transformations

四类变换

- ▶ **对数**: $\log(1 + x)$ 压缩右尾, 系数可读作近似弹性;
- ▶ **标准化 z-score**: 均值 0、标准差 1, 跨变量可比;
- ▶ **归一化 min-max**: 映射到 $[0, 1]$, 受极值影响大;
- ▶ **分箱**: cut (等宽) 与 qcut (等频), 把连续变量变类别。

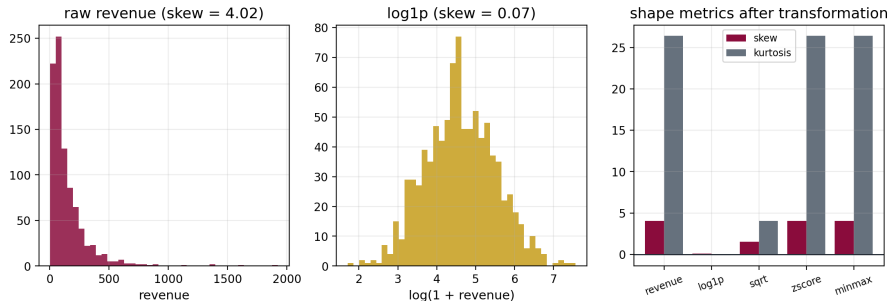
选择依据

1. 分布形状 (偏度): 右偏明显 $\rightarrow$ 先试对数;
2. 量纲是否可比: 多变量入模 $\rightarrow$ 标准化;
3. 是否需要可解释的系数: 对数变换优于标准化;
4. 是否存在真实极端值: 先处理异常, 再决定变换。

提醒: 变换**改变了变量的含义**, 报告里要写清“因变量是 $\log$ 收入”。

来源: 变换清单为通用实践; 本讲样本的偏度与峰度变化见 72 页图。

【样本 + 可视化】变换与标准化效果 Taming Skew



左：原始营收（偏度 ≈ 2.7 ，右偏严重）；中： $\log(1 + x)$ 后接近对称；右：各变换后的偏度与峰度对比——分箱与标准化不改变分布形状，只改变尺度。

程序 13_10_transform_assert.py · 样本 13_income_skewed.csv · 汇总表 13_transform_summary.csv

单位与量纲统一 Uniformity of Units

原课件的第五项质量特征

Uniformity: 数据用**同一计量单位**表示的程度。单位不统一时, "1.2" 可能是元、千元或百万元。

常见单位陷阱

元/万元/亿元·美元/人民币 (还要定汇率口径) · 吨/千克·百分比 vs 百分点·名义 vs 实际 (是否剔通胀)

工程做法

1. 把单位**单独成列**, 值保持数值型;
2. 定义**基准单位**与换算函数 (在代码里, 不在脑子里);
3. 断言: 换算后同一实体跨源差异应在容差内;
4. 字段字典里写明单位、口径与汇率来源。

与"唯一性 Uniqueness" 并列: 本讲把原课件的 Uniformity 与大纲的唯一性都纳入质量框架 (8-9 页)。

来源: Uniformity 表述沿用原课件第 3 页; 工程做法为本讲补充。

【代码】变换 + 断言管道 Transform and Validate

```
1 transformed["log1p"] = np.log1p(transformed["revenue"]) # log(1 + x) compresses the tail
2 transformed["sqrt"] = np.sqrt(transformed["revenue"]) # square root: a gentler compression
3 transformed["zscore"] = ((transformed["revenue"] - transformed["revenue"].mean()) # centre
4                          / transformed["revenue"].std(ddof=0)) # scale to mean 0, sd 1
5 transformed["quartile"] = pd.qcut(transformed["revenue"], 4, # equal-count bins
6                                   labels=["Q1", "Q2", "Q3", "Q4"]) # readable labels
7 summary = pd.DataFrame([{"variable": n, "skew": round(transformed[n].skew(), 3)} # shape
8                          for n in ["revenue", "log1p", "sqrt", "zscore", "minmax"]]) # five columns
9 RULES = {"firm_id": {"unique": True, "regex": r"~F\d{4}$"}, # key: unique and patterned
10         "region": {"allowed": ["East", "Central", "West"]}, # category: closed domain
11         "revenue": {"min": 0.0, "max": 1_000_000.0, "nullable": False}} # numeric contract
12 report = check(transformed, RULES) # run the contract, collect violations
13 print("schema checks passed:", int(report["ok"].sum()), "of", len(report)) # 12 of 12
14 assert report["ok"].all(), "schema contract failed: fix the table before publishing" # gate
```

这就是“最小可用数据契约”：字段存在性、唯一性、正则、取值域、非空**逐条断言**，失败**即停**——生产环境里同类的工具是 **pandera** 与 **Great Expectations** (78 页)。

Stata 命令 → pandas 对照 (原课件第 30 页)

Stata 命令 (原课件)	pandas 等价
drop ... if ... / keep ... if ... drop(keep) ... if ... (按条件筛子集)	df = df[cond] / df = df[~cond] query() 或布尔索引
replace ... if ...	df.loc[cond, "col"] = value
gen(egen) ... if ...	df["new"] = ... / groupby().transform()
order ... , before(last)	df.reindex(columns=[...])
label variable ...	字段字典 / 数据契约 (列名即标签)
rename ...	df.rename(columns={...})
merge 1:1 / m:1 ... using ...	merge(validate="1:1" / "m:1")
append using ...	pd.concat([...])
joinby ...	merge(on=group) (行数相乘, 慎用)
xtset id t	df.groupby("id")["t"] 覆盖检查
ipolate ... , epolate	interpolate(limit_direction="both")

来源: 命令清单沿用原课件第 30 页 (Stata Commends); pandas 对照为本讲补充, 代码页 42-74 页均可用 Python 完成。

pandas 速查 (原课件第 29、35–37 页)

合并与比较

`merge` `join` `concat` `compare` `combine_first` `update`; pandas 为索引与关系代数提供各类集合逻辑 (并集/交集)。

重塑与透视

`pivot` `pivot_table` `stack/unstack` `melt` `wide_to_long` `get_dummies/from_dummies` `explode` `crosstab` `cut` `factorize`

来源：资料链接与主题沿用原课件第 29 页与 pandas 官方文档的 Merge/Reshape 两章。

原课件给的资料入口

- ▶ pandas 官方 *10 Minutes to pandas* 入门指南;
- ▶ Visual Python: 图形化界面生成 pandas 代码;
- ▶ 中文实战教程 (知乎专栏)。

本讲把原课件的资料入口升级为**可运行样本**: 10 个程序 + 39 份样本数据, 全部逐行英文注释。

大表整备：Polars 与 DuckDB Handling Bigger Tables

Polars

列式内存布局 + 惰性执行 (LazyFrame) :
先写完整转换计划再一次性优化执行; 多线程、内存占用低, API 与 pandas 相近 (select/filter/group_by/join)。

DuckDB

嵌入式列式数据库: 直接对 CSV/Parquet 写 SQL, 无需导入; 适合“用 SQL 做整备”的团队与超大文件的聚合。

何时切换

- ▶ 数据能放进内存 (本讲样本级别) → pandas 足够;
- ▶ 单机内存吃紧、上亿行 → Polars / DuckDB;
- ▶ 要多人协作、有调度与血缘需求 → 进入数仓/湖仓 (79 页)。

原则: 先让流程正确, 再让流程变快; 用 Parquet 落盘既省空间又保留类型。

来源: Polars 与 DuckDB 的定位据其官方文档 (2026); 与 L1 现代数据栈、L2 开放表格式呼应。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 变换、工具

与工程化 Transformation, Tools and Engineering

可发现管道: pandera 与 Great Expectations Data Contracts in Practice

pandera

用 Python 声明式描述 schema: 列名、类型、取值域、唯一性、类别闭集、跨列规则; 与 pandas/Polars 无缝集成, 可在函数入口**自动校验**。

Great Expectations

“期望套件”式校验: expect_column_values_to_not_be_null 等开箱规则, 生成**数据文档** (Data Docs), 可放进 CI 或调度任务里定期跑。

落地方式

1. 把契约写进**代码仓库** (与数据同版本);
2. 每次落盘前校验, 失败**即停并告警**;
3. 校验结果存档, 作为质量报告的证据;
4. 契约变更走**评审** (与其他代码一样)。

本讲代码页 74 页用 20 行实现的断言管道, 就是这两个工具的最小内核。

来源: pandera 与 Great Expectations 官方文档 (2026)。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整備 | **变换、工具与工程化** Transformation, Tools and Engineering

dbt 与特征存储 From Scripts to Platforms

dbt: 把转换工程化

每个模型一个 SQL 文件 (select 即定义), 自动处理依赖顺序; 内置测试 (唯一、非空、关系)、文档与血缘图; 支持增量物化与快照。

特征存储 Feature Store

解决“训练用的特征”与“线上推理用的特征”不一致 (training-serving skew): 统一的特征定义 + 离线表 + 在线表, 保证同一份口径两侧复用。

与本讲连接

本讲的“分层 + 契约 + 断言”就是这两类平台的最小内核; 平台帮你规模化, 但口径与质量判断仍然是人做的。

取舍: 课程项目规模用脚本 + 契约 + 断言足够; 只有在多人协作、频繁变更时才需要平台。

来源: dbt 与特征存储 (Feast 等) 官方文档 (2026); 与 L1 现代数据栈、L5 MLOps 衔接。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 变换、工具

与工程化 Transformation, Tools and Engineering

LLM 辅助清洗 LLM-Assisted Cleaning (2026 新增)

适合交给 LLM 的部分

- ▶ **长尾类别归并**: 上千种职位/行业名 → 标准类目;
- ▶ **地址与公司名标准化**、"简称 → 全称";
- ▶ **非结构化文本抽取**: 把备注字段变成结构化列;
- ▶ **异常解释**: 为可疑记录给出可能的原因假设。

三条纪律

1. **规则优先**: 能用映射表解决的不要用模型;
2. **抽样人工核验**: 随机抽 30-50 条与人工标注比对一致率;
3. **提示词与模型留档**: 模型版本、提示词、温度、运行日期写进报告。

风险提示: LLM 会**自信地编造**归一结果 (幻觉); 对金额、代码、ID 一类字段, 不要让模型"顺手改"。与 L2 的 LLM 抽取、L11 的大模型内容衔接。

脱敏与隐私增强 De-identification and Privacy

识别与处理 PII

姓名、身份证、手机号、邮箱、地址、银行卡、设备号 → **识别** → **处理** → **记录**。处理方式：删除、**假名化**（加盐哈希，可关联不可反查）、泛化（年龄段/城市级）。

更强的保护

- ▶ **k-匿名**：每条记录至少与 $k-1$ 条不可区分；
- ▶ **差分隐私**：对统计结果加噪声，给出隐私预算 ϵ ；
- ▶ **合成数据**：保留统计特征、不保留真实个体。

合规底线（与 L2 合规红线一致）：个人信息最小必要、目的限定、**不把未脱敏数据放进公开仓库**；跨境传输与二次分发要单独评估。

来源：PII 处理与隐私增强技术据 PIPL/GDPR 要求与隐私工程实践整理。

数据漂移监控与回滚 Drift Monitoring and Rollback

两类漂移

- ▶ **模式漂移** *schema drift*: 列被改名/删除、类型变化、枚举新增 → 断言会直接失败;
- ▶ **分布漂移** *distribution drift*: 缺失率、均值、类别占比缓慢变化 → 需要监控指标。

监控什么

缺失率·唯一键重复率·数值分位数·类别占比·PSI / KL 散度·上游接口成功率与延迟。每个指标配一条**阈值与动作** (告警 / 阻断发布)。

回滚

快照 + 版本化脚本 = 一键回到上一版; 分析表可重建, 不用重新抓数。

与本讲的连接

14 页的”快照不可变 + 幂等落盘”就是回滚能力的前提。

来源: 漂移监控指标与回滚流程据数据工程实践整理; 与 L1 数据管道、L5 MLOps 监控衔接。

大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 变换、工具

与工程化 Transformation, Tools and Engineering

小结：把整備做成工程 Preparation as Engineering

一条流水线

契约 → 断言 → 日志 → 监控 → 回滚：每加一个数据源，就把这五件事补齐一次，而不是“这次先手工处理一下”。

三层成熟度

1. 能跑：脚本化、可重跑；
2. 可信：断言 + 质量报告 + 审计日志；
3. 可运营：契约版本化、漂移监控、自动回滚。

课程项目的目标是把第 1-2 层做扎实：脚本 + 断言 + 报告，第 3 层在项目中加分。

项目 1「数据获取与整备」要求 Project 1 Requirements

交付物 (四件)

1. **整备脚本**: 从原始数据到分析表可一键重跑 (含断言与日志);
2. **清洗后的数据集**: raw / curated 分层, 含 filled_flag 等标记列;
3. **数据说明**: 字段字典、单位、口径、缺失与插补处理;
4. **质量报告**: 五维评分 + 处理清单 + 影响说明 (86 页模板)。

评分维度

需求理解·数据处理规范性·可复现性 (能否被别人重跑)·结果解释·报告表达·AI 使用声明。(满分 100 分)

常见失分点: 没有日志、缺失处理没写清、图表与结论对不上、脚本在别人机器上跑不起来。

来源: 与课程大纲第六节项目说明一致; 组队不超过 2 人, 允许自选数据并鼓励结合 AI 工具 (须声明与核验)。

一页选题说明（第 6 周前提交） One-Page Proposal

要素	写什么
问题 Question	一句话研究问题 + 为什么重要（商业/金融/贸易/营销场景）
数据 Data	来源、覆盖范围、时间跨度、获取方式与许可；已知的质量问题
方法 Method	整备流程（合并/清洗/缺失/变换）+ 计划的分析或模型方法
预期产出 Output	交付物清单（脚本、数据集、报告）+ 预期结论形态
风险 Risks	数据拿不到、口径不一致、样本太小等风险与备选方案

写作建议：一页 A4、五段式，每段 3-5 句；把”数据”段落写清楚比把”方法”写漂亮更重要。

来源：与课程大纲”第 6 周前提交一页选题说明”的要求一致。

数据质量报告模板 Quality Report Template

第一部分：五维评分

有效性 / 准确性 / 完整性 / 一致性 / 单位一致 (或唯一性) —— 每项给出分数 + 计算口径 + 最差的字段。

第二部分：处理清单

每步一行：步骤·影响行数·依据·责任人 (即 13_cleaning_log.csv)。

一页纸即可；关键是每个论断后面都有数字，而不是“数据质量较好”这类形容词。

来源：模板据本讲的样本页与代码页 (15、41、46、52 页) 输出整理。

第三部分：量化结论

缺失率与模式·重复率·异常率与处置·类别归并前后变体数·单位换算影响。

第四部分：影响与局限

处理对结论的潜在影响、未解决的遗留问题、下一步计划。

清洗与整备清单 Checklist (30 项)

读入与结构

☐ 指定编码与 dtype; ☐ 保留原始快照; ☐ 明确粒度 (一行代表什么); ☐ 主键唯一性断言; ☐ 时间列可解析; ☐ 分层目录 raw/curated

缺失

☐ 缺失率与模式; ☐ 机制判断 (MCAR/MAR/M-NAR); ☐ 处理策略与依据; ☐ 插补值打标记; ☐ 敏感性分析

合并与重塑

☐ schema 对齐 (改名/类型/单位); ☐ 基数声明 validate; ☐ indicator 对账; ☐ 行数守恒; ☐ source 标记; ☐ 宽长转换往返无损

面板

☐ $(id, time)$ 唯一; ☐ 缺口与 T_i ; ☐ 个体进出原因; ☐ 覆盖率表

清洗

☐ 完全重复; ☐ 主键重复规则; ☐ 不相关观测; ☐ 类别映射表; ☐ 文本脏数据; ☐ 异常值分类 (不可能 vs 统计)

变换与交付

☐ 单位统一; ☐ 分布与标准化; ☐ 字段字典; ☐ 断言全通过; ☐ 质量报告; ☐ AI 使用声明

工具与命令速查 Toolbox

任务	工具 / 关键 API
表内整備	pandas: merge / concat / pivot / melt / groupby / fillna / interpolate
大表整備	Polars (LazyFrame) · DuckDB (对 Parquet 写 SQL) · Parquet 落盘
契约与校验	pandera · Great Expectations · 自写断言 (本讲 74 页)
转换工程化	dbt (模型 + 测试 + 文档 + 血缘) · 特征存储 (Feast)
交互式清洗	OpenRefine (少量数据、人工核对类别) · Jupyter + pandas
面板与统计	statsmodels / linearmodels (面板固定效应) · Stata (xtset/ipolate)
监控	Evidently / whylogs (分布漂移) · 调度告警 (Airflow / Dagster)

来源：工具据课程大纲第 3 讲“工具与数据”栏 (pandas、Polars、DuckDB、OpenRefine、SQL) 与 2026 年工程实践整理。

术语对照 Glossaries

中文	English
有效性	Validity
准确性	Accuracy
完整性	Completeness
一致性	Consistency
单位一致 / 唯一性	Uniformity / Uniqueness
粒度	Grain
追加 / 合并	Append / Merge
连接基数	Cardinality
重塑	Reshaping

中文	English
缺失机制	MCAR / MAR / MNAR
插补 / 多重插补	Imputation / Multiple imputation
插值 / 外推	Interpolation / Extrapolation
平衡面板	Balanced panel
覆盖率	Coverage rate
缩尾	Winsorise
数据契约	Data contract
漂移	Drift
回滚	Rollback

来源：术语取自本讲正文与原课件用词，便于阅读英文文献与文档。

参考书与资料 References

数据整备与清洗

- ▶ McKinney, W. *Python for Data Analysis*, 3rd ed., Ch.7 Data Cleaning and Preparation、Ch.8 Data Wrangling: Join, Combine, Reshape (课程指定教材, 对应大纲 L3 阅读)
- ▶ pandas User Guide: *Merge, join, concatenate and compare*、*Reshaping and pivot tables*、*Working with missing data*
- ▶ Rahm & Do (2000) Data cleaning: problems and current approaches; Kim et al. (2003) taxonomy of dirty data

缺失数据与面板

- ▶ Little & Rubin, *Statistical Analysis with Missing Data*
- ▶ van Buuren, *Flexible Imputation of Missing Data*
- ▶ Enders, *Applied Missing Data Analysis*
- ▶ Stata 手册: merge / append / joinby / xtset / ipolate (含 gymdata、ipolxmpl1 官方示例)

工程文档: pandera ·Great Expectations ·dbt ·Feast ·Polars ·DuckDB ·Evidently (均为 2026 年在线文档)。

来源: 教材与文献对应课程大纲 L3 的“工具与数据”与阅读要求; 工程文档为 2026 年版本。
大数据分析 [2026 国际商务] Big Data Analytics | Lecture 3 Data Preparation 第三讲: 数据整备 | 项目与结语

Projects and Wrap-up

检查点：让同学复现你的数据 Reproducibility Drill

课堂演练 (15 分钟)

1. 交换脚本 + 原始数据，不给任何口头说明；
2. 对方在自己的电脑上重跑；
3. 对比：行数、列数、关键列的均值/缺失率；
4. 若不一致 → 找出哪一步依赖了隐含信息（本地文件、手工改动、未固定的顺序）。

常见“复现失败”原因

未固定随机种子·依赖本地文件路径·用手工 Excel 改过数据·未锁定依赖版本·依赖字典顺序（应为列名访问）·时区/编码差异

本讲所有程序都通过了这项检验：`python run_all.py` 在任意机器上得到完全相同的样本与图。

来源：复现演练设计对应本讲“可复现五原则”（14 页）与项目 1 的评分维度。

谢谢！

Thank you ·Q & A

本讲小结：质量五特征 → 合并与重塑 → 清洗五步法 → 缺失机制与插补 → 面板平衡性 → 变换与工程化

样本与程序：website/code/lecture03 (10 个程序，逐行英文注释)

website/data/lecture03 (39 份样本) website/figures/lecture03 (12 张插图)

任课教师：戴谢尔 (Xieer Dai, David) ·Xieer.dai@szu.edu.cn ·四方楼南 209